

Topology Optimization

Lecture 5 - Constrained Optimization 2

Sequential Quadratic Programming

Method of Moving Asymptotes

MPI and PETSc

Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction

Some mathematical prerequisites ✓ **DONE!**

Mathematical and numerical formulation of the Finite Element Method ✓ **DONE!**

Optimization Theory and numerical optimization methods

Basic Topology Optimization

Advanced Topology Optimization

Applications of Topology Optimization

Steepest Descent (from last week)

The most basic gradient-based method for choosing the descent direction is termed steepest descent or gradient descent.

The descent direction is chosen such that a small step, $\alpha^k > 0$, in the $\mathbf{d}^k$ direction, $\mathbf{x}^k + \alpha^k \mathbf{d}^k$, will decrease the cost function.

The *steepest descent direction* is $-\nabla f(\mathbf{x}^k)$ and

$$f(\mathbf{x}^k + \alpha^k \mathbf{d}^k) < f(\mathbf{x}^k) \Rightarrow f(\mathbf{x}^k) + \alpha^k \nabla f(\mathbf{x}^k) \cdot \mathbf{d}^k < f(\mathbf{x}^k) \Rightarrow \nabla f(\mathbf{x}^k) \cdot \mathbf{d}^k = -|\nabla f(\mathbf{x}^k)|^2 < 0$$

where a Taylor expansion and $\mathbf{d} = -\nabla f(\mathbf{x}^k)$ was used.

The algorithm is therefore:

Choose a starting point, $\mathbf{x}^0$.

Set $k = 0$.

While (Not converged) and ($k < k_{max}$)

Calculate $\nabla f(\mathbf{x}^k)$.

$\mathbf{d}^k = -\nabla f(\mathbf{x}^k)$.

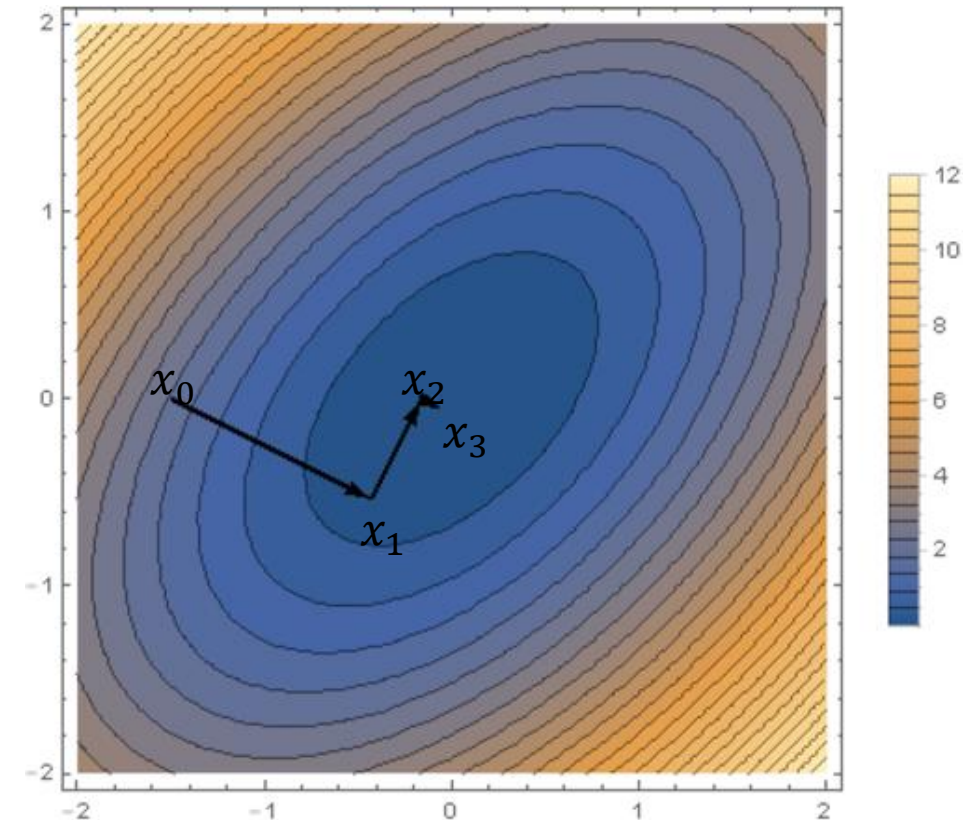
Find α^k by minimizing $f(\mathbf{x}^k + \alpha^k \mathbf{d}^k)$.

$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{d}^k$.

$k = k + 1$.

Convergence can be checked with e.g. $\|\nabla f(\mathbf{x}^k)\| < \epsilon_1$ and/or monitoring the change in $\mathbf{x}$, $\|\mathbf{x}^k - \mathbf{x}^{k-1}\| < \epsilon_2$.
 ϵ_i are tolerances.

For the most very basic implementation, the step size α^k can be a constant or adjusted heuristically and the 1D minimization is not needed.



(from last week)

THEOREM 4.6

Karush-Kuhn-Tucker Optimality Conditions

Let $\mathbf{x}^*$ be a regular point of the feasible set that is a local minimum for $f(\mathbf{x})$, subject to $h_i(\mathbf{x}) = 0; i = 1$ to $p; g_j(\mathbf{x}) \leq 0; j = 1$ to m . Then there exist Lagrange multipliers $\mathbf{v}^*$ (a p -vector) and $\mathbf{u}^*$ (an m -vector) such that the Lagrangian function is stationary with respect to x_j, v_i, u_j , and s_j at the point $\mathbf{x}^*$.

1. Lagrangian function for the problem written in the standard form:

$$\begin{aligned} L(\mathbf{x}, \mathbf{v}, \mathbf{u}, \mathbf{s}) &= f(\mathbf{x}) + \sum_{i=1}^p v_i h_i(\mathbf{x}) + \sum_{j=1}^m u_j (g_j(\mathbf{x}) + s_j^2) \\ &= f(\mathbf{x}) + \mathbf{v}^T \mathbf{h}(\mathbf{x}) + \mathbf{u}^T (\mathbf{g}(\mathbf{x}) + \mathbf{s}^2) \end{aligned} \quad (4.46)$$

2. Gradient conditions:

$$\frac{\partial L}{\partial x_k} = \frac{\partial f}{\partial x_k} + \sum_{i=1}^p v_i^* \frac{\partial h_i}{\partial x_k} + \sum_{j=1}^m u_j^* \frac{\partial g_j}{\partial x_k} = 0; \quad k = 1 \text{ to } n \quad (4.47)$$

$$\frac{\partial L}{\partial v_i} = 0 \Rightarrow h_i(\mathbf{x}^*) = 0; \quad i = 1 \text{ to } p \quad (4.48)$$

$$\frac{\partial L}{\partial u_j} = 0 \Rightarrow (g_j(\mathbf{x}^*) + s_j^2) = 0; \quad j = 1 \text{ to } m \quad (4.49)$$

3. Feasibility check for inequalities:

$$s_j^2 \geq 0; \text{ or equivalently } g_j \leq 0; \quad j = 1 \text{ to } m \quad (4.50)$$

4. Switching conditions:

$$\frac{\partial L}{\partial s_j} = 0 \Rightarrow 2u_j^* s_j = 0; \quad j = 1 \text{ to } m \quad (4.51)$$

5. Nonnegativity of Lagrange multipliers for inequalities:

$$u_j^* \geq 0; \quad j = 1 \text{ to } m \quad (4.52)$$

6. *Regularity check*: Gradients of the active constraints must be linearly independent. In such a case the Lagrange multipliers for the constraints are unique.

Alternative form of KKT conditions (from last week)

TABLE 5.1 Alternate form of KKT necessary conditions

Problem: Minimize $f(\mathbf{x})$ subject to $h_i(\mathbf{x}) = 0, i = 1$ to p ; $g_j(\mathbf{x}) \leq 0, j = 1$ to m

1. Lagrangian function definition
$$L = f + \sum_{i=1}^p v_i h_i + \sum_{j=1}^m u_j g_j \quad (5.1)$$

2. Gradient conditions
$$\frac{\partial L}{\partial x_k} = 0; \quad \frac{\partial f}{\partial x_k} + \sum_{i=1}^p v_i^* \frac{\partial h_i}{\partial x_k} + \sum_{j=1}^m u_j^* \frac{\partial g_j}{\partial x_k} = 0; \quad k = 1 \text{ to } n \quad (5.2)$$

3. Feasibility check
$$h_i(\mathbf{x}^*) = 0; \quad i = 1 \text{ to } p; \quad g_j(\mathbf{x}^*) \leq 0; \quad j = 1 \text{ to } m \quad (5.3)$$

4. Switching conditions
$$u_j^* g_j(\mathbf{x}^*) = 0; \quad j = 1 \text{ to } m \quad (5.4)$$

5. Non-negativity of Lagrange multipliers for inequalities
$$u_j^* \geq 0; \quad j = 1 \text{ to } m \quad (5.5)$$

6. Regularity check: Gradients of active constraints must be linearly independent. In such a case, the Lagrange multipliers for the constraints are unique.

The slack variables have been eliminated in (4.51) using (4.49), $g_j(\mathbf{x}^*) = -s_j^2$, and one works directly with the inequality constraint functions, $g_j(\mathbf{x}^*)$, in the feasibility check.

Sequential Linear Programming (SLP) (from last week)

$$\begin{aligned} & \min_{\mathbf{x}} f(\mathbf{x}) \text{ such that} \\ & h_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \\ & g_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, m \end{aligned}$$

Can be replaced by twice as many inequality constraints:

$$\begin{aligned} h_i(\mathbf{x}) &\leq 0, \\ -h_i(\mathbf{x}) &\leq 0 \end{aligned}$$

Set a maximum constraint violation ϵ_1 , a minimum move ϵ_2 and a maximum number of steps k_{max} .

Choose an initial design $\mathbf{x}^0$.

$k = 0$.

While ($k = 0$ or $g_i > \epsilon_1$ or $|h_i| > \epsilon_1$ or $|\mathbf{d}|^k > \epsilon_2$) and $k < k_{max}$:

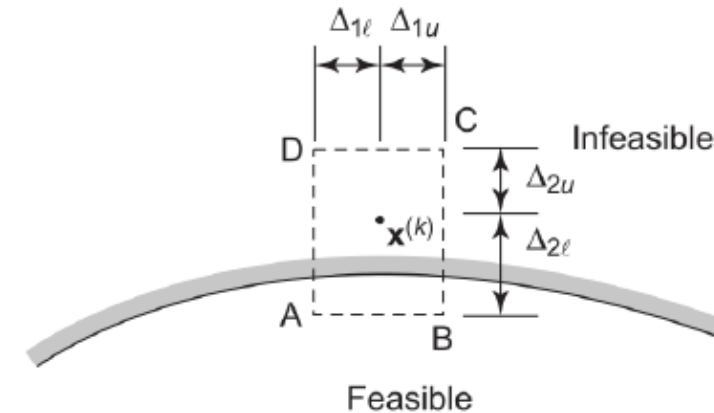
Linearize the problem around $\mathbf{x}^k$.

Select move limits Δ_{il}^k and Δ_{iu}^k for all design variables.

Solve the linear programming subproblem augmented with move limits to obtain $\mathbf{d}^k$.

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{d}^k.$$

$$k = k + 1.$$



The linear sub-problem can be solved with special methods for linear problems (e.g. the Simplex method).

The hard part is to choose appropriate move limits that are not too loose and not too tight.

Sequential Linear Programming works well for improving a design, but should not be used as a general "black-box" solver for non-linear constrained optimization problems due to the deficiencies highlighted above.

Today we discuss sequential quadratic programming and the method of moving asymptotes. Both use some of the ideas from sequential linear programming and works extremely well.

We will mainly use the method of moving asymptotes in this course.

Quadratic Programming

A quadratic programming problem have a quadratic cost function and linear constraints.

The general quadratic programming problem can be written as:

$$\begin{aligned} &\text{Minimize } q(\mathbf{d}) = \mathbf{c}^T \mathbf{d} + \frac{1}{2} \mathbf{d}^T \mathbf{H} \mathbf{d} \text{ with respect to } \mathbf{d} \\ &\text{subject to } \mathbf{N}^T \mathbf{d} = \mathbf{e} \\ &\quad \mathbf{A}^T \mathbf{d} \leq \mathbf{b} \end{aligned}$$

where

$\mathbf{d}$ is the vector of n unknown

$\mathbf{c}$ is a vector of n constants

$\mathbf{b}$ is a vector of m constants

$\mathbf{e}$ is a vector of p constants

$\mathbf{H}$ is a $n \times n$ matrix of constants

$\mathbf{A}$ is a $n \times m$ matrix of constants

$\mathbf{N}$ is a $n \times p$ matrix of constants

If the problem has a solution and $\mathbf{H}$ is positive definite the problem is strictly convex and has a unique global minimum.

If the problem has a solution and $\mathbf{H}$ is positive semidefinite the problem is convex and any local minimum is also a global minimum, but it may not be unique.

Descent Function

- For unconstrained problems we can monitor the progress of an algorithm by looking at the change in cost function value.
- In constrained optimization one can usually not use the cost function to monitor the descent towards a minimum, so a descent function is defined.
- When a search direction have been found, the descent function is minimized along this direction to find the step size.
- An algorithm is named “globally convergent” if it finds a *local* minimum from any initial values of the design variables.
- For an algorithm to be globally convergent it must be true that:
 - A descent function is defined, and it decreases in each iteration.
 - The step direction is a continuous function of the design variables.
 - The feasible set is closed and bounded.
 - The set is closed if there are no strict inequalities in the problem.
 - The set is bounded if we have upper and lower limits on all design variables.
 - This is most often the case in engineering problems.
- Sequential Linear Programming is not globally convergent ☹
- Sequential Quadratic Programming (usually) is ☺

Sequential Quadratic Programming (SQP)

Set a maximum constraint violation ϵ_1 , a minimum move ϵ_2 , and a maximum number of steps k_{max} .

Choose an initial design $\mathbf{x}^0$.

$k = 0$.

While ($k = 0$ or $g_i > \epsilon_1$ or $|h_i| > \epsilon_1$ or $\|\alpha^{k-1}\mathbf{d}^{k-1}\| > \epsilon_2$) and $k < k_{max}$:

 Define a quadratic programming subproblem around $\mathbf{x}^k$.

 Solve the quadratic programming subproblem to obtain the search direction $\mathbf{d}^k$.

 Minimize the descent function along $\mathbf{d}^k$ to obtain the step size α^k .

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{d}^k$$

$$k = k + 1$$

No move limits are used, instead a linesearch (1-dimensional minimization) is performed to determine the step size.

Different SQP methods can be made by:

1. Using different quadratic subproblems.
2. Using different method to solve the quadratic subproblem.
3. Using different descent functions.
4. Using different methods to find the step size.

Constrained Steepest Descent

For Constrained Steepest Descent, the quadratic subproblem at iteration k is defined as

$$\begin{aligned} &\text{Minimize } \bar{f}(\mathbf{d}) = \mathbf{c}^T \mathbf{d} + \frac{1}{2} \mathbf{d}^T \mathbf{d} \text{ with respect to } \mathbf{d} \\ &\text{subject to } \mathbf{N}^T \mathbf{d} = \mathbf{e} \quad \text{and} \quad \mathbf{A}^T \mathbf{d} \leq \mathbf{b} \end{aligned}$$

where

$$\begin{aligned} \mathbf{c} &= \nabla f(\mathbf{x}^k) \quad , \quad \mathbf{e} = -\mathbf{h}(\mathbf{x}^k) \quad , \quad \mathbf{b} = -\mathbf{g}(\mathbf{x}^k) \quad , \\ \mathbf{N} &= \frac{\partial \mathbf{h}(\mathbf{x}^k)}{\partial \mathbf{x}} \quad , \quad \mathbf{A} = \frac{\partial \mathbf{g}(\mathbf{x}^k)}{\partial \mathbf{x}} \end{aligned}$$

The cost function is strictly convex ($\mathbf{H} = \mathbf{I}$) and the problem will then have a unique global minimum, if it exist.

If no constraints are active at the minimum, $\mathbf{d}$ is found from

$$\frac{\partial \bar{f}}{\partial d_i} = c_i + d_i = 0 \quad \Rightarrow \quad \nabla f(\mathbf{x}^k) + \mathbf{d} = 0 \quad \Rightarrow \quad \mathbf{d} = -\nabla f(\mathbf{x}^k)$$

This is just the steepest descent direction and the reason for the name of the methods.

The solution of the quadratic programming problem will determine the search direction.

Compared to sequential linear programming, we do not have to worry about move limits.

The Constrained Steepest Descent algorithm can be proven to be globally convergent.

Using 2nd order information

The general quadratic programming sub-problem in iteration k can be written as:

$$\begin{aligned} \text{Minimize } q(\mathbf{d}) &= \mathbf{c}^T \mathbf{d} + \frac{1}{2} \mathbf{d}^T \mathbf{H} \mathbf{d} & \text{where } \mathbf{c} &= \nabla f(\mathbf{x}^k) \quad , \quad \mathbf{e} = -\mathbf{h}(\mathbf{x}^k) \quad , \quad \mathbf{b} = -\mathbf{g}(\mathbf{x}^k) \quad , \\ \text{s.t. } \mathbf{N}^T \mathbf{d} &= \mathbf{e} & \mathbf{N} &= \frac{\partial \mathbf{h}(\mathbf{x}^k)}{\partial \mathbf{x}} \quad , \quad \mathbf{A} = \frac{\partial \mathbf{g}(\mathbf{x}^k)}{\partial \mathbf{x}} \\ \mathbf{A}^T \mathbf{d} &\leq \mathbf{b} \end{aligned}$$

- If $\mathbf{H}$ is positive definite the problem is strictly convex and has a unique global minimum.
- Constrained Steepest Descent used $\mathbf{H}=\mathbf{I}$, which lead to an algorithm like steepest descent when no constraints are active.
- If one takes $\mathbf{H}$ as the Hessian matrix, then then the algorithm will behave like Newton's method when no constraints are active.
 - Newton's method for unconstrained optimization is much faster than steepest descent in terms of number of iterations needed.
 - The Hessian matrix is, however, not guaranteed to be positive definite.
 - So-called quasi-Newton methods for unconstrained optimization uses a positive definite matrix to approximate the Hessian.
- In Sequential Quadratic Programming this idea can also be used.
- A widely used algorithm in this class is SNOPT (Wikipedia 3/4 2019, <https://en.wikipedia.org/wiki/SNOPT>):
 - SNOPT, for Sparse Nonlinear OPTimizer, is a software package for solving large-scale nonlinear optimization problems written by Philip Gill, Walter Murray and Michael Saunders. SNOPT is mainly written in Fortran, but interfaces to C, C++, Python and MATLAB are available.
 - It employs a sparse sequential quadratic programming (SQP) algorithm with limited-memory quasi-Newton approximations to the Hessian of the Lagrangian. It is especially effective for nonlinear problems with functions and gradients that are expensive to evaluate. The functions should be smooth but need not be convex.
- SNOPT can easily be used for Topology Optimization, e.g., in COMSOL.
- We will, however, use the Method of Moving Asymptotes which solves the problems with a method similar to Sequential Linear Programming, but with the huge benefit that it actually works very well.

(Globally Convergent) Method of Moving Asymptotes ((GC)MMA)

- In SLP:
 - Cost function and constraint functions are linearized and move limits are added
 - Tight move limits will limit the step size but ensures the linear functions are accurate approximations.
 - Loose move limits make a big step possible, but convergence issues might appear due to the break-down of the linear approximations.
- In MMA:
 - Functions are 1st order approximations but include terms like $1/(u-x)$ or $1/(x-l)$ instead of x (as in SLP).
 - u and l are known as the moving asymptotes.
 - The asymptotes act as move limits (since, e.g., $1/(u-x)$ become large when $x \rightarrow u$) and they change in each iteration.
 - Inner iterations can be included to make the algorithm globally convergent. This is named the GCMMA method.
 - Can be formulated for a quite general type of constrained problem, which can be used to solve:
 - Generally constrained problems.
 - Min-max (“worst-case”) problems.
 - Least square problems. (we will not need this functionality, though).
- MMA comes in the two versions, GCMMA and MMA.
- GCMMA is the latest version which is, as the name suggests, globally convergent meaning that it will converge to a solution no matter the starting point used.
- GCMMA adds “inner” iterations to MMA to make it globally convergent.
- Choosing to do no inner iterations, GCMMA is $\sim$ MMA.
- We will often say MMA to denote both MMA and GCMMA...
- The MMA implementation found on Brightspace is CGMMA.

Method of Moving Asymptotes (MMA)

MMA solves the optimization problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & f_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

The design variables are in the vector $\mathbf{x}$. The vector $\mathbf{y}$ and the scalar z are "artificial" variables.

a_0 , a_i , c_i and d_i are real valued parameters which must satisfy $a_0 > 0$, $a_i \geq 0$, $c_i \geq 0$, $d_i \geq 0$, $c_i + d_i > 0$ for $i = 1, \dots, m$ and $a_i c_i > a_0$ for all $a_i > 0$.

MMA minimizes a cost function under general constraints and has bound constraints on all the x_j variables.

Many algorithms support bound constraints, not only MMA, as they are often easy to handle.

The artificial variables make it easy to state different kinds of optimization problems.

Scaling of variables and functions is important in numerics. As we saw last week, scaling the functions by positive factors does not change the solution. If possible, functions and variables should be scaled according to:

- The cost function should be scaled such that $1 \leq f_0(\mathbf{x}) \leq 100$. Does not change the solution, x_j .
- If $g_i(\mathbf{x}) \leq g_i^{\max}$, let $f_i(\mathbf{x}) = s_i (g_i(\mathbf{x}) - g_i^{\max})$ and adjust s_i until $1 \leq s_i g_i^{\max} \leq 100$. Does not change the solution, x_j .
- The variables should be scaled to satisfy $0.1 \leq x_j^{\max} - x_j^{\min} \leq 100$. Changes the solution, x_j , but easy to "scale back".

Method of Moving Asymptotes (MMA)

MMA solves the optimization problem

$$\min_{\mathbf{x}, \mathbf{y}, z} f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right)$$

$$\text{subject to } f_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m, \quad x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n, \quad \mathbf{y} \geq 0, \quad z \geq 0$$

a_0, a_i, c_i and d_i are real values parameters which must satisfy $a_0 > 0, a_i \geq 0, c_i \geq 0, d_i \geq 0, c_i + d_i > 0$ for $i = 1, \dots, m$ and $a_i c_i > a_0$ for all $a_i > 0$.

(Things above the dashed line is a recap from previous slides...)

Choosing $a_0 = 1, a_i = 0, d_i = 1$ and c_i "a large number", we have

$$\min_{\mathbf{x}, \mathbf{y}, z} f_0(\mathbf{x}) + z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} y_i^2 \right)$$

$$\text{subject to } f_i(\mathbf{x}) - y_i \leq 0, \quad i = 1, \dots, m, \quad x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n, \quad \mathbf{y} \geq 0, \quad z \geq 0$$

Standard Constrained
Optimization Problem

Then z is independent of everything else, which yield a minimum at $z = 0$, as $z \geq 0$ is required.

Since the c_i 's are large, having $y_i > 0$ will add a large value to the cost function so $\mathbf{y} \approx 0$ will most likely appear in a solution.

This is then almost equivalent to the problem

$$\min_{\mathbf{x}} f_0(\mathbf{x})$$

$$\text{subject to } f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \quad x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n$$

which is "the standard constrained optimization problem". If some $y_i > 0$, it signals that a feasible solution could not be found.

Method of Moving Asymptotes (MMA)

MMA solves the optimization problem

$$\min_{\mathbf{x}, \mathbf{y}, z} f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right)$$

subject to $f_i(\mathbf{x}) - a_i z - y_i \leq 0, i = 1, \dots, m, x_j^{\min} \leq x_j \leq x_j^{\max}, j = 1, \dots, n, \mathbf{y} \geq 0, z \geq 0$

a_0, a_i, c_i and d_i are real values parameters which must satisfy $a_0 > 0, a_i \geq 0, c_i \geq 0, d_i \geq 0, c_i + d_i > 0$ for $i = 1, \dots, m$ and $a_i c_i > a_0$ for all $a_i > 0$.

(Recap)

Choosing $f_0(\mathbf{x}) = 0, f_i(\mathbf{x}) = h_i(\mathbf{x})$ for $i = 1, \dots, p, f_{i+p} = g_i(\mathbf{x})$ for $i = 1, \dots, q, a_0 = a_1 = \dots = a_p = 1, a_{p+1} = \dots = a_{p+q} = 0, d_1 = \dots = d_m = 1$ and $c_1 = \dots = c_m =$ "a large number" we get

$$\min_{\mathbf{x}, \mathbf{y}, z} z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right)$$

subject to $h_i(\mathbf{x}) - y_i \leq z, i = 1, \dots, p,$

$g_i(\mathbf{x}) - y_i \leq 0, i = 1, \dots, q, x_j^{\min} \leq x_j \leq x_j^{\max}, j = 1, \dots, n, \mathbf{y} \geq 0, z \geq 0$

Min-max optimization problem.

Since c_i are large, we expect to get a solution with $\mathbf{y} = 0$ and the MMA problem is almost equivalent to

$$\min_{\mathbf{x}, z} z$$

subject to $h_i(\mathbf{x}) \leq z, i = 1, \dots, p,$

$g_i(\mathbf{x}) \leq 0, i = 1, \dots, q, x_j^{\min} \leq x_j \leq x_j^{\max}, j = 1, \dots, n, z \geq 0$

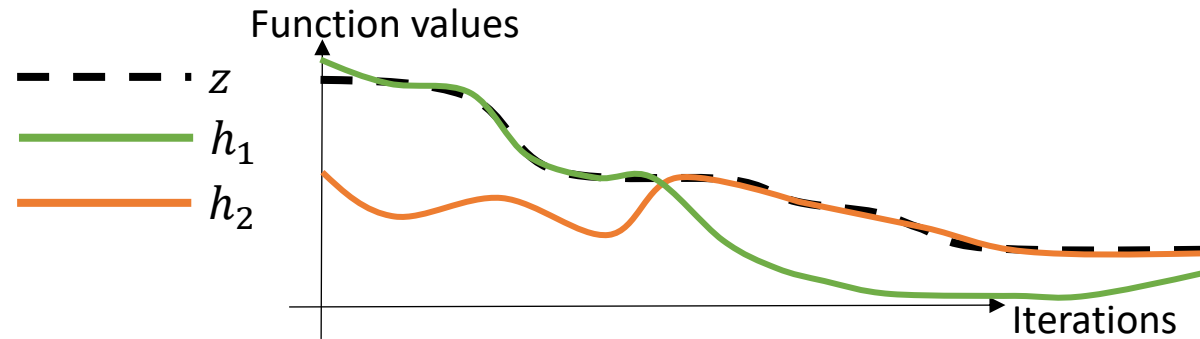
Method of Moving Asymptotes (MMA)

In the problem

$$\begin{aligned} \min_{\mathbf{x}, z} \quad & z \\ \text{subject to} \quad & h_i(\mathbf{x}) \leq z, \quad i = 1, \dots, p, \\ & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q, \quad x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n, \quad z \geq 0 \end{aligned}$$

Min-max optimization problem.

the variable z acts as an upper bound to the constraints $h_i(\mathbf{x})$ if it is required that $h_i(\mathbf{x}) \geq 0$ since $z \geq 0$.



Since $z \geq 0$, we should make sure $h_i(\mathbf{x}) \geq 0$ always, e.g. by adding the same constant to all h_i functions.

This problem is a way of solving a so-called "min-max" problem

$$\begin{aligned} \min_{\mathbf{x}} \quad & \max_{i=1, \dots, p} \{h_i(\mathbf{x})\} \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q, \quad x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n, \quad z \geq 0 \end{aligned}$$

If, e.g., the functions $h_i(\mathbf{x}) \geq 0$ represents cost functions for a number of different designs, the maximum among the h_i 's represent the worst design.

The min-max problem thus try to make the worst-case as good as possible.

Method of Moving Asymptotes (MMA)

MMA solves the optimization problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & f_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

a_0, a_i, c_i and d_i are real values parameters which must satisfy $a_0 > 0, a_i \geq 0, c_i \geq 0, d_i \geq 0, c_i + d_i > 0$ for $i = 1, \dots, m$ and $a_i c_i > a_0$ for all $a_i > 0$.

(Recap)

Choosing $f_0(\mathbf{x}) = 0, f_i(\mathbf{x}) = h_i(\mathbf{x})$ for $i = 1, \dots, p, f_{i+p}(\mathbf{x}) = -h_i(\mathbf{x})$ for $i = 1, \dots, p, f_{2p+i}(\mathbf{x}) = g_i(\mathbf{x})$ for $i = 1, \dots, q, a_0 = 1, a_1 = \dots = a_{2p+q} = 0, d_1 = \dots = d_m = 1, c_1 = \dots = c_{2p} = 0$ and $c_{2p+1} = \dots = c_{2p+q} = \text{"a large number"}$ we get

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & z + \sum_{i=1}^p \frac{1}{2} (y_i^2 + y_{p+i}^2) + \sum_{i=1}^q \left(c_{2p+i} y_{2p+i} + \frac{1}{2} y_{2p+i}^2 \right) \\ \text{subject to} \quad & h_i(\mathbf{x}) \leq y_i, \quad i = 1, \dots, p \\ & -h_i(\mathbf{x}) \leq y_{p+i}, \quad i = 1, \dots, p \\ & g_i(\mathbf{x}) \leq y_{2p+i}, \quad i = 1, \dots, q \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

Least-squares
optimization problem
(for reference, not used
in this course)

Method of Moving Asymptotes (MMA)

The problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & z + \sum_{i=1}^p \frac{1}{2} (y_i^2 + y_{p+i}^2) + \sum_{i=1}^q \left(c_{2p+i} y_{2p+i} + \frac{1}{2} y_{2p+i}^2 \right) \\ \text{subject to} \quad & h_i(\mathbf{x}) \leq y_i, \quad i = 1, \dots, p \\ & -h_i(\mathbf{x}) \leq y_{p+i}, \quad i = 1, \dots, p \\ & g_i(\mathbf{x}) \leq y_{2p+i}, \quad i = 1, \dots, q \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

Least-squares
optimization problem
(for reference, not used
in this course)

is expected to have a solution with $z = 0$ and $y_{2p+i} = 0$ for $i = 1, \dots, q$.

The first two sets of inequalities can also be written as $-y_{p+i} \leq h_i(\mathbf{x}) \leq y_i$.

Since y_i and y_{p+i} enter in the same squared way in the cost function, we expect $-y_{p+i} = y_i = h_i(\mathbf{x})$.

The MMA problem is then almost equivalent to the least squares problem

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_{i=1}^p h_i^2(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \end{aligned}$$

Method of Moving Asymptotes (MMA)

MMA solves the optimization problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & f_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

(Recap)

a_0, a_i, c_i and d_i are real values parameters which must satisfy $a_0 > 0, a_i \geq 0, c_i \geq 0, d_i \geq 0, c_i + d_i > 0$ for $i = 1, \dots, m$ and $a_i c_i > a_0$ for all $a_i > 0$.

To actually solve the optimization problem, MMA creates an approximate optimization problem by expanding all f_i functions near $\mathbf{x}^k$ as

$$f_i(\mathbf{x}) \approx \tilde{f}_i(\mathbf{x}) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j} + \frac{q_{ij}^k}{x_j - l_j} \right) + r_i \quad i = 1, \dots, m$$

$$p_{ij}^k = (u_j - x_j^k)^2 \left(1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$q_{ij}^k = (x_j^k - l_j)^2 \left(0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$r_i^k = f_i(\mathbf{x}^k) - \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j^k} + \frac{q_{ij}^k}{x_j^k - l_j} \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ = \max \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- = \max \left(-\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right)$$

Theory we will not go through in detail....

The u_j 's and the l_j 's are named the "moving asymptotes". ρ_i^k are positive constants. k is an iteration index. $x_j^{\min} \leq x_j \leq x_j^{\max}$ and $l_j < x_j < u_j$.

Method of Moving Asymptotes (MMA)

To actually solve the optimization problem, MMA creates an approximate optimization problem by expanding all f_i functions near $\mathbf{x}^k$ as

$$f_i(\mathbf{x}) \approx \tilde{f}_i(\mathbf{x}) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j} + \frac{q_{ij}^k}{x_j - l_j} \right) + r_i \quad i = 1, \dots, m$$

$$p_{ij}^k = (u_j - x_j^k)^2 \left(1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$q_{ij}^k = (x_j^k - l_j)^2 \left(0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$r_i^k = f_i(\mathbf{x}^k) - \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j^k} + \frac{q_{ij}^k}{x_j^k - l_j} \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ = \max \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- = \max \left(-\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right)$$

(Recap)

The u_j 's and the l_j 's are named the "moving asymptotes". ρ_i^k are positive constants. k is an iteration index. $x_j^{\min} \leq x_j \leq x_j^{\max}$ and $l_j < x_j < u_j$.

Notice that p_{ij}^k and q_{ij}^k are always positive and the approximate functions are all convex:

A function $f(\mathbf{x})$ defined on a convex set S is convex *if and only if* the Hessian matrix of the function is positive semidefinite or positive definite at all points in S .

Theory we will not go through in detail....

$$\frac{\partial \tilde{f}_i(\mathbf{x})}{\partial x_t} = \sum_{j=1}^n \left(\frac{p_{ij}^k}{(u_j - x_j)^2} \frac{\partial x_j}{\partial x_t} - \frac{q_{ij}^k}{(x_j - l_j)^2} \frac{\partial x_j}{\partial x_t} \right) = \frac{p_{it}^k}{(u_t - x_t)^2} - \frac{q_{it}^k}{(x_t - l_t)^2}$$

$$H_{st} = \frac{\partial^2 \tilde{f}_i(\mathbf{x})}{\partial x_s \partial x_t} = 2 \left(\frac{p_{it}^k}{(u_t - x_t)^3} + \frac{q_{it}^k}{(x_t - l_t)^3} \right) \delta_{st} > 0 \quad \Rightarrow \quad \text{The Hessian is diagonal with all positive elements}$$

Method of Moving Asymptotes (MMA)

$$f_i(\mathbf{x}) \approx \tilde{f}_i(\mathbf{x}) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j} + \frac{q_{ij}^k}{x_j - l_j} \right) + r_i \quad i = 1, \dots, m$$

$$p_{ij}^k = (u_j - x_j^k)^2 \left(1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$q_{ij}^k = (x_j^k - l_j)^2 \left(0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right)$$

$$r_i^k = f_i(\mathbf{x}^k) - \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j^k} + \frac{q_{ij}^k}{x_j^k - l_j} \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ = \max \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right) \quad , \quad \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- = \max \left(-\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right)$$

(Recap)

By doing a linear Taylor expansion of $\tilde{f}_i$ around $\mathbf{x}^k$ we get

$$\tilde{f}_i(\mathbf{x}) \approx \tilde{f}_i(\mathbf{x}^k) + \nabla \tilde{f}_i(\mathbf{x}^k) \cdot (\mathbf{x} - \mathbf{x}^k)$$

$$\tilde{f}_i(\mathbf{x}^k) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j^k} + \frac{q_{ij}^k}{x_j^k - l_j} \right) + r_i = f_i(\mathbf{x}^k)$$

$$\begin{aligned} \nabla \tilde{f}_i(\mathbf{x}^k) \cdot (\mathbf{x} - \mathbf{x}^k) &= \sum_{j=1}^m \frac{\partial \tilde{f}_i(\mathbf{x}^k)}{\partial x_j} (x_j - x_j^k) = \sum_{j=1}^m \left(\frac{p_{ij}^k}{(u_j - x_j^k)^2} - \frac{q_{ij}^k}{(x_j^k - l_j)^2} \right) (x_j - x_j^k) \\ &= \sum_{j=1}^m \left(\left(1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right) - \left(0.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 1.001 \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i^k}{x_j^{\max} - x_j^{\min}} \right) \right) (x_j - x_j^k) \\ &= \sum_{j=1}^m \left(\left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^+ - \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} \right)^- \right) (x_j - x_j^k) = \sum_{j=1}^m \left(\max \left(\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right) - \max \left(-\frac{\partial f_i(\mathbf{x}^k)}{\partial x_j}, 0 \right) \right) (x_j - x_j^k) = \sum_{j=1}^m \frac{\partial f_i(\mathbf{x}^k)}{\partial x_j} (x_j - x_j^k) \\ &= \nabla f_i(\mathbf{x}^k) \cdot (\mathbf{x} - \mathbf{x}^k) \end{aligned}$$

Theory we will not go through in detail....

And thus $\tilde{f}_i(\mathbf{x}) \approx f_i(\mathbf{x}^k) + \nabla f_i(\mathbf{x}^k) \cdot (\mathbf{x} - \mathbf{x}^k)$, showing $\tilde{f}_i(\mathbf{x})$ is a linear approximation to $f_i(\mathbf{x})$ near $\mathbf{x}^k$.

Summary of CGMMA

MMA solves the optimization problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & f_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & f_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \end{aligned}$$

Adjusting the parameters a_i , c_i and d_i , the solution of this problem is equivalent to solutions of some common optimization problems ("normal", "worst case"/"min-max" and "least square").

In each "outer" iteration of the algorithm ($k = 0, 1, \dots$), MMA forms and solves a number of convex sub-problems, the "inner" iterations, where the sub-problems become more and more conservative to ensure global convergence.

The inner sub-problem are

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & \tilde{f}_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & \tilde{f}_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & \alpha_j \leq x_j \leq \beta_j, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \\ & \tilde{f}_i(\mathbf{x}) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j} + \frac{q_{ij}^k}{x_j - l_j} \right) + r_i \quad i = 1, \dots, m \end{aligned}$$

The constant value, r_0 , in $\tilde{f}_0$ can be dropped.

The bounds and moving asymptotes satisfy $x_j^{\min} \leq \alpha_j \leq x_j \leq \beta_j \leq x_j^{\max}$ and $l_j < \alpha_j \leq x_j \leq \beta_j < u_j$.

In each "inner" step of the algorithm, it solves the sub-problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}, z} \quad & \tilde{f}_0(\mathbf{x}) + a_0 z + \sum_{i=1}^m \left(c_i y_i + \frac{1}{2} d_i y_i^2 \right) \\ \text{subject to} \quad & \tilde{f}_i(\mathbf{x}) - a_i z - y_i \leq 0, \quad i = 1, \dots, m \\ & \alpha_j \leq x_j \leq \beta_j, \quad j = 1, \dots, n \\ & \mathbf{y} \geq 0, \quad z \geq 0 \\ & \tilde{f}_i(\mathbf{x}) = \sum_{j=1}^n \left(\frac{p_{ij}^k}{u_j - x_j} + \frac{q_{ij}^k}{x_j - l_j} \right) + r_i \quad i = 1, \dots, m \end{aligned}$$

(Recap)

Since all functions are convex, this optimization sub-problem is convex and a local solution is also a global solution.

For a convex optimization problem, the KKT conditions are necessary *AND* sufficient conditions for an optimum. The inner workings of MMA solves these KKT conditions.

The solution of the sub-problem is made efficient and parallelizable by the fact that each x_j each appear in only one term in the sum in $\tilde{f}_i(\mathbf{x})$.

k is named the "outer" iteration index.

In "ordinary" MMA, the sub-problem is solved once and the solution becomes $\mathbf{x}^{k+1}$.

In GCMMA, a series of the sub-problems are solved in "inner" iterations where $\mathbf{x}^k$ is not updated.

In each inner iteration, new and more "conservative" function approximations are made until $\tilde{f}_i^k(\mathbf{x}^k) \geq f_i(\mathbf{x}^k)$.

The more conservative approximations are made by updating ρ^k in p_{ij}^k and q_{ij}^k . These inner iterations leads to the global convergence of GCMMA.

When the inner iterations are done, the solution of the last (and most conservative) sub-problem becomes the next $\mathbf{x}^k$.

Summary of CGMMA

Method of Moving Asymptotes (MMA)

- Update formulas for the different variables and details on how to solve the sub-problem can be found in the Svanberg paper.
- Some adjustable parameters enter the update formulas and have default values:
 - How α_i and β_i are calculated:
 - albefa=0.1
 - move=0.5
 - How the moving asymptotes are updated:
 - asyinit=0.5
 - asydecr=0.7
 - asyincr=1.2
- Some more parameters are needed for the total solution process:
 - How accurate the inner optimization problems are solved: muTol= 10^{-6} .
 - Maximum number of iterations for solving the inner problem: llmax=1000.
 - Maximum number of inner iterations pr. outer iteration: lmax=10.
 - Maximum number of outer iterations: kmax=100.
 - Minimum number of outer iterations: kmin=0.
 - Outer tolerances on x_i and $f_i(\mathbf{x})$: xtol=0.01 and ftol=0.001.
- It is most common to play with:
 - lower “move” to take smaller steps, which can be useful for difficult optimization problems.
 - “kmax” and “lmax” to adjust the maximum number of outer and inner steps.
 - “xtol” and “ftol” to adjust the accuracy of the solution.

Method of Moving Asymptotes (MMA)

- MMA is implemented in the file `mma.py`.
- Two versions are implemented, one using `numpy` and one using `PETSc`.
- `PETSc` is the linear algebra backend of `FEniCS` and allows one to solve problems using many cores.
- If you want to look deeply into the code, the `numpy` is most easy to understand, but we will use the `PETSc` version with `FEniCS`.
- The interface to each version is very similar.
- Some tests and examples are in the file `mma_tests.zip`.
- The `mma.py` code is a bit messy, sorry about that 😊

```
class MMA_petsc:
    """
    Method of Moving Asymptotes using numpy
    min f[0]+a0*z+1/2*b0*z**2+sum_{i=1}^m (c[i]*y[i]+1/2*d[i]*y[i]**2)
    s.t.
    f[i]-a[i]*z-y[i] <= 0
    z>=0, y[i]>=0
    xmin[j]<=x_j<=xmax[j]

    Keep d[i]>0 and b0>0 always!
    """
```

Method of Moving Asymptotes (MMA)

The examples in `mma_test.zip` are in the files `test*.py`. Put these files in a directory and copy `mma.py` to that directory also.

Each file has a “main part”, which creates and runs a test/example using `mma`.

For example, from `test4pet.py` :

```
if __name__ == "__main__":
    import sys
    np.set_printoptions(linewidth=180)

    # Get rank of CPU core the code is running on
    rank=PETSc.Comm(PETSc.COMM_WORLD).getRank()

    # Print stuff only on CPU core 0
    if rank==0:
        print('Running test problem 4 using PETSc')
```

```
Imports in mma.py:
from petsc4py import PETSc
import numpy as np
import scipy as sp
import scipy.sparse
import time
import sys
```

The ability to run problems using more cores come from the MPI framework, used in PETSc.

MPI is short for “Message Passing Interface” and is a standard library for creating parallel scientific applications.

To execute the program in parallel using 2 cores, write

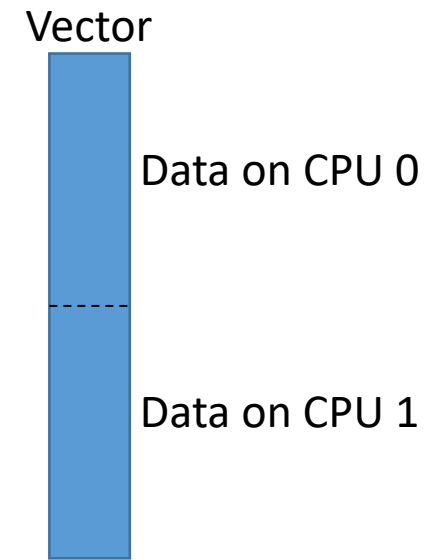
```
mpiexec -n 2 python3 test4pet.py
```

The command ‘`python3 test4pet.py`’ is executed two times and each instance runs the same code in the python file.

When a parallel PETSc vector or matrix is created, each instance will only hold a part of the vector or matrix.

Each CPU will in vector and matrix operations only work on part of the problem, thus speeding up the execution.

If the vector and/or matrices are small, communication overhead between the CPUs will destroy the benefit of parallelization.



Method of Moving Asymptotes (MMA)

Since the program is now executed more than once, print statements needs to be executed only on one of the CPU's.

Try running

```
mpiexec -n 2 python3 -c 'print("Hello")'
```

The print can be controlled using the 'rank', which is a unique number for each process/CPU.

For example, from test4pet.py:

```
if __name__ == "__main__":
    import sys
    np.set_printoptions(linewidth=180)

    # Get rank of CPU core the code is running on
    rank=PETSc.Comm(PETSc.COMM_WORLD).getRank()

    # Print stuff only on CPU core 0
    if rank==0:
        print('Running test problem 4 using PETSc')

    # Create initial guess as a PETSc vectore with 10000 1s.
    x0=PETSc.Vec().createMPI(10000)
    x0.setValues(range(0,10000),np.ones(10000))
    x0.assemble()

    # Create instance of problem class
    t4p=test_problem4_pet(x0)
```

Method of Moving Asymptotes (MMA)

The interface to the numpy and PETSc versions is similar, as you will notice by browsing the test cases/examples.

The PETSc test no. 4 is in the test4pet.py file and implemented as

```
class test_problem4_pet():
    """
    | min sum x_i^2 s.t. sum x_i = 0, (sum x_i)**2 = 0, 0<= x_i <= 2
    | solution x_i = 0
    | Since x_i>=0, equality constraints are implemented simply as
    | inequality constraints (<=0).
    """
    def __init__(self,x0):
        # Create instance of MMA
        self.mma=MMA_petsc(x0,2,f=self.f,g=self.g)
        # Set lower bound of variables
        self.mma.xmin[:]=0.0
        # Set upper bound of variables
        self.mma.xmax[:]=2.0
        # Get number of variables from MMA
        self.n=self.mma.n
        # Store exact solution for comparison
        self.exact=np.zeros(x0.size)

    def f(self,x):
        # Define cost function and constraint functions
        return np.array([ x.dot(x)/self.n**2 , x.sum()/self.n , x.sum()**2/self.n**2 ])

    def g(self,x):
        # Return part of the Jacobian of f stored on the CPU core
        tmp=np.zeros(self.mma.nlocal)
        tmp[:]=2*x.getArray()[:]
        tmp2=np.ones(self.mma.nlocal)
        tmp3=2*x.sum()*tmp2
        return np.array([ tmp/self.n**2 , tmp2/self.n , tmp3/self.n**2 ])
```

In the main part

```
# Create initial guess as a PETSc vectore with 10000 1s.
x0=PETSc.Vec().createMPI(10000)
x0.setValues(range(0,10000),np.ones(10000))
x0.assemble()

# Create instance of problem class
t4p=test_problem4_pet(x0)

# Some lines skipped...

# Create instance of tester class
tester=MMA_tester(t4p.exact)

# Test solution
test=tester.test(sol4p)

# Transfer solution so it is present on all CPU cores
local_sol=t4p.mma.parToLocal(sol4p)

# Print stuff only on CPU core 0
if rank==0:
    print('Solution',local_sol)
    print('PETSc test problem 4 done! Passed:',test)
    print()
```

- min-max problem:

- $\min \max (x_1, x_2, x_3) \text{ s.t. } x_1 + x_2 + x_3 = 15$

- Solution ?

Method of Moving Asymptotes (MMA)

The problem is implemented in test7pet.py as

```
class test_problem7_pet():
    """
    | solves a min-max problem:
    |   min max (x1,x2,x3) s.t. x1 + x2 + x3 = 15
    """
    def __init__(self,x0):
        # 5 constraints, since 3 min-max functions + 2 constraints
        self.mma=MMA_petsc(x0,5,f=self.f,g=self.g)
        self.mma.xmin[:]=-100
        self.mma.xmax[:]=100
        self.mma.setMinMax(3,2) # 3 min-max functions and 2 constraints
        self.exact=np.array([5,5,5])

    def f(self,x):
        y=self.mma.parToLocal(x)
        # Keep min-max functions >0
        c=50
        delta=1e-8
        ar=np.array([ 0 , y[0]+c , y[1]+c , y[2]+c ,
                     y[0]+y[1]+y[2]-15-delta , -y[0]-y[1]-y[2]+15-delta ])
        return ar

    def g(self,x):
        ar=np.array([ [0,0,0] , [1,0,0] , [0,1,0] , [0,0,1] ,
                     [1,1,1] , [-1,-1,-1] ])
        return ar[:,self.mma.nlow:self.mma.nhigh]
```

From the main part:

```
if rank==0:
    print('Running test problem 7 using PETSc')
    x0=PETSc.Vec().createMPI(3)
    x0.setValues(range(0,3),[0,0,0])
    x0.assemble()
    t7p=test_problem7_pet(x0)

    # Test gradients with finite differences
    t7p.mma.testGrads(components=[0,1,2])

    sol7=t7p.mma.solve(x0)
    tester=MMA_tester(t7p.exact)
    test=tester.test(sol7) # Test solution
    local_sol=t7p.mma.parToLocal(sol7)
    if rank==0:
        print('Solution',local_sol)
        print('PETSc test problem 7 done! Passed:',test)
        print()
```

Remember to call mma's
setMinMax when solving
min-max problems with mma.py.
This will set the parameters of
mma as previously described in
the theory section for min-max.